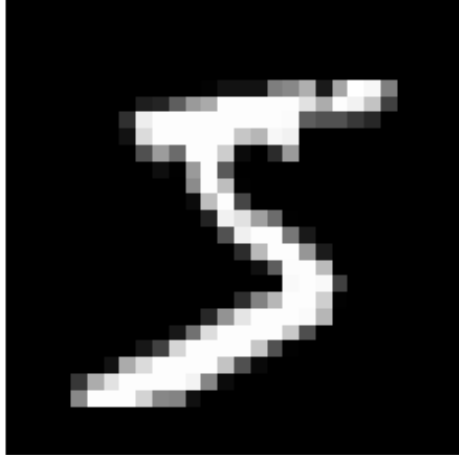
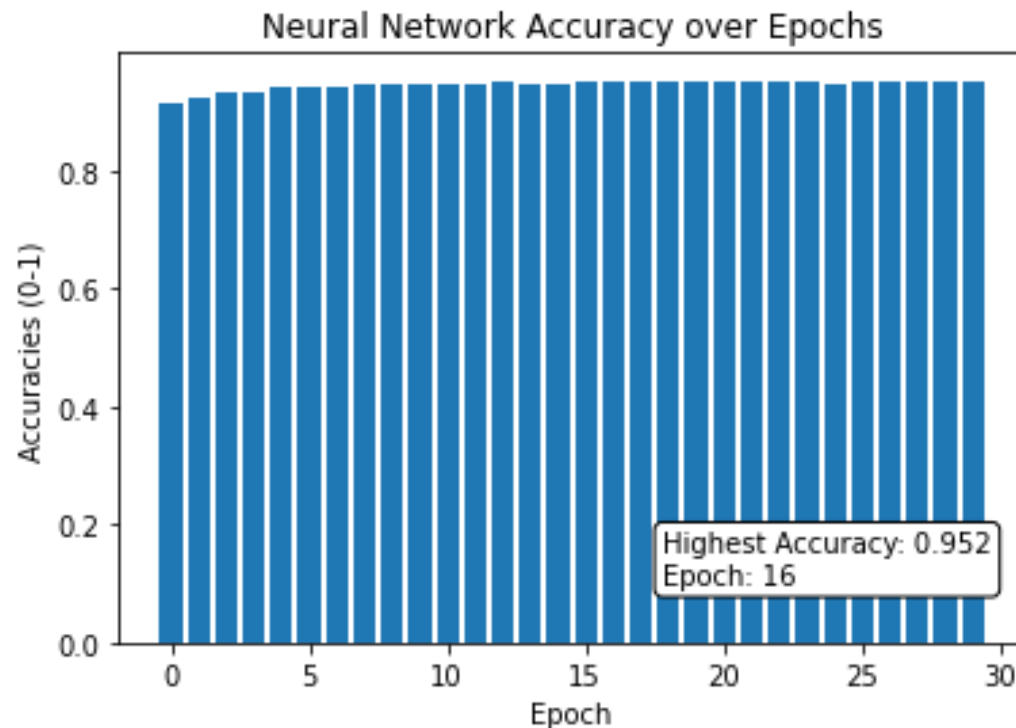
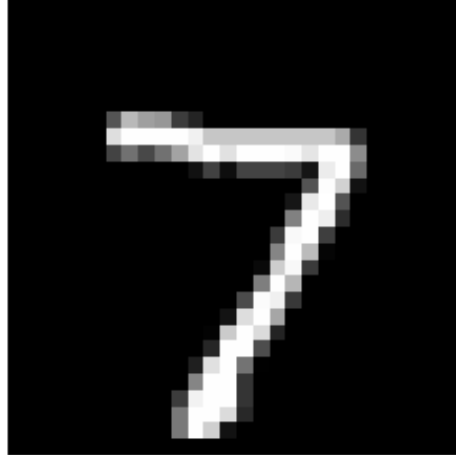


First MNIST Training Data - Class: 5



First MNIST Testing Data - Class: 7



Machine Learning: Neural Networks using a Multilayer Perceptron

Objectives

- To determine on how to extend the mechanisms of a perceptron in making a neural network
- To train a neural network that can classify the NMIST dataset up to a 95% accuracy

NMIST and the Input Layer

In previous exercises, we trained a perceptron and used logistic regression as classifiers – the former with a binary nature and the latter a continuous one. An alternative way of looking at these models is as a neural network with N input layers, a single neuron in the hidden layer representing the single activation function, and an output neuron.

Hence in making a full neural network, what we do is add more neurons to the hidden layer, and then add the desired amount of neurons in the output layer as seen in Fig 1.

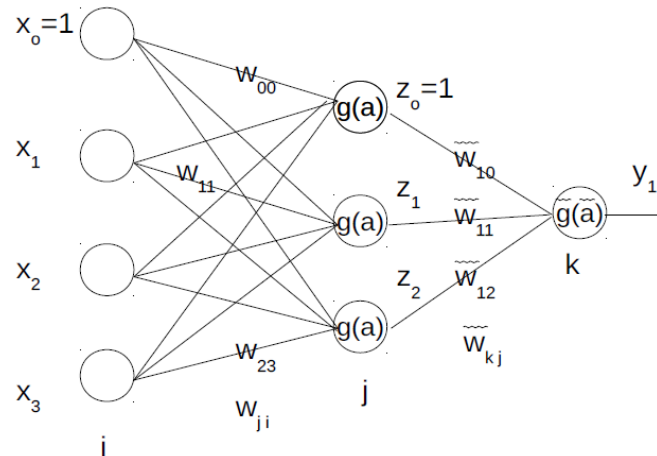


Fig 1. A sample neural network with 4 neurons in the input, 3 in the hidden layer, and one output. [1]

The input layer consists of normalized parameters that we want to use in order to train our network. In making a perceptron we had two neurons in the input layer that held the two features I've chosen namely eccentricities and mean grayscales of our fruit data. The same was done for the input layer of the previous logistic regression. However for this activity, we do something much more complicated...

The NMIST is a commonly used database for evaluating the performance of neural networks. It consists of handwritten digits in a 24×24 grid correctly classified by humans. We use all 60,000 training images and 10,000 test images in training our neural network. A sample of the dataset is shown below in Fig 2.

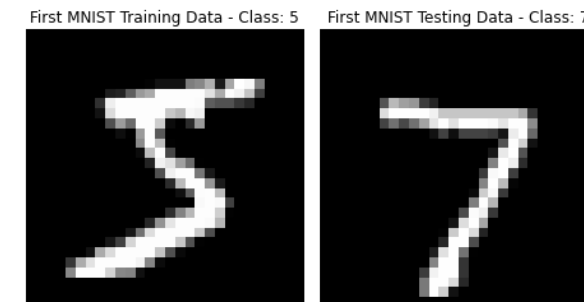


Fig 2. The first training and testing image in the NMIST dataset [2]

In training the neural network, we treat each pixel in this grid as a neuron in the input layer. Hence, we can think of the model as taking into account 784 features for training.

Training

In this particular example, I decided to feed this input layer to one single hidden layer comprising 30 neurons. This choice is completely arbitrary but since it produced the accuracy I want we're sticking with it.

To derive the algorithm we will use, assume we have a neural networks with hidden layers and with corresponding weights and biases in each layer. I have written it in LaTeX because using Powerpoint would be weird...

Consider a single training example with a one-hot encoded classification. Let this correct classification be y . We compare this to a neural network with L layers such that the vector representing the activations of the output layer is a^L , and the one previous to it is a^{L-1} , a^{L-2} , ... which stops of course at the input layer. We define the cost function for a single training image to be

$$C_x = (a^L - y)^2 \quad (1)$$

and the activation of our current layer a^L is ofcourse

$$a^L = \sigma(w^L a^{L-1} + b^L) \quad (2)$$

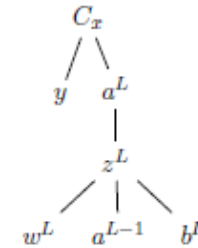
$$\equiv \sigma(z^L) \quad (3)$$

where

$$w^L = \text{weight matrix that dictates } a^L \quad (4)$$

$$b^L = \text{biases stored in vector} \quad (5)$$

We then have the following tree of dependencies



for a layer a^L . Note that we can naturally do this recursively for the previous layer a^{L-1} and so on. Hence, in order to minimize the cost function we take the following derivative

$$\frac{\partial C_x}{\partial w^L} = \frac{\partial z^L}{\partial w^L} \frac{\partial a^L}{\partial z^L} \frac{\partial C_x}{\partial a^L} \quad (6)$$

we can calculate each of these derivatives easily

$$\frac{\partial C_x}{\partial a^L} = 2(a^L - y) \quad (7)$$

$$\frac{\partial a^L}{\partial z^L} = \text{derivative of sigmoid} \quad (8)$$

$$= \sigma'(z^L) \quad (9)$$

$$\frac{\partial z^L}{\partial w^L} = a^{L-1} \quad (10)$$

We then calculate Equation 6 for all weights and biases, this gives us the gradient of the cost function. Finally, subtract this gradient to all the weights to update them. For biases, note that

$$\frac{\partial C_x}{\partial b^L} = \sigma'(z^L) 2[a^L - y] \quad (11)$$

The algorithm detailed above was derived from the output layer, but you only need to change indices to update the weights for previous layers! This is called **backpropagation**. The only difference is that we must note that

$$\frac{\partial C_x}{\partial a^{L-1}} = w^L \sigma'(z^L) 2[a^L - y] \quad (12)$$

To shorten runtimes, I also utilized **Stochastic Gradient Descent (SGD)** which basically just translates to dividing the training data into batches, then using backpropagation on each.

Performance

The initial weights are assigned at random and are adjusted through our weight change rule. Hence even if we construct the same network without changing any parameters, accuracies may still vary – although these differences are negligible for practical purposes. **In Fig 3. below I show the performance of the same network that was trained for 30 epochs:**

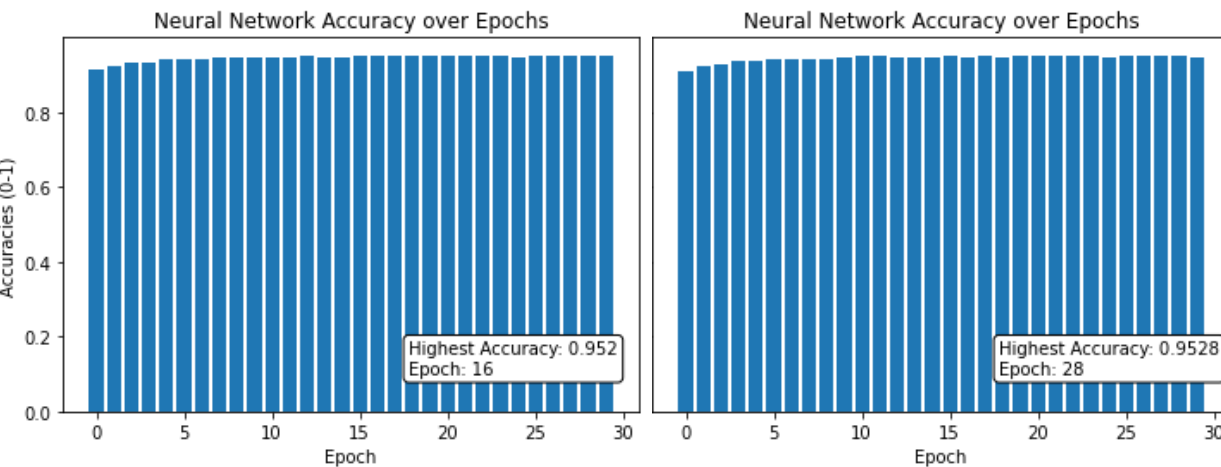


Fig 3. A neural network with the number of neurons in each layer being [784, 30, 10], a batch size of 10 for SGD, and learning rate of 3.0

We can see that the initial conditions of a neural network can dictate how fast it can converge to the most accurate solution. Let's see what happens if we add more layers. In my code implementation, this is really easy to do!

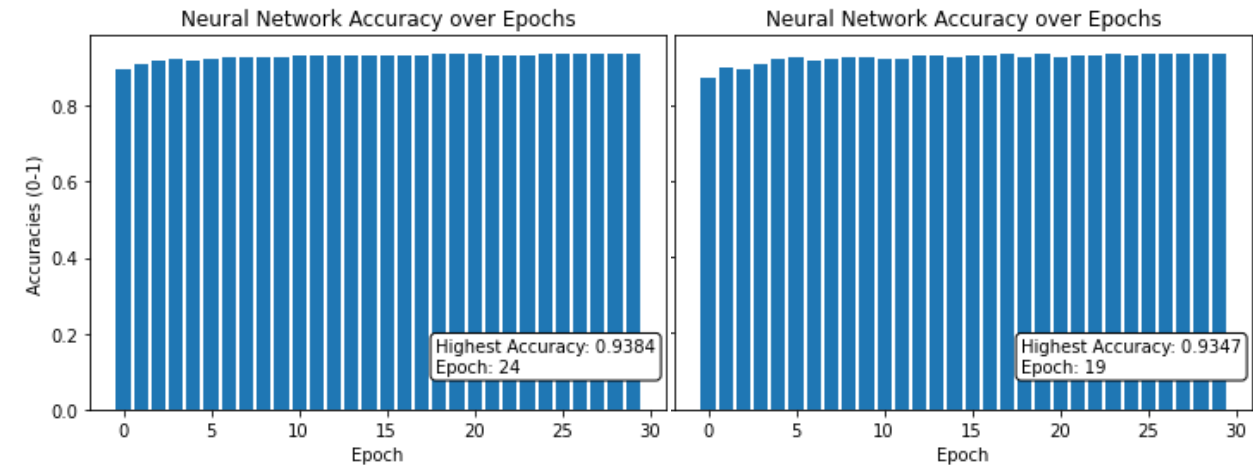


Fig 4. Two neural networks with layers having neurons [784, 16, 16, 10] and [784, 16, 16, 16, 10] respectively, with the same batch size and learning rate.

We can see that adding more layers doesn't translate to a higher accuracy. We need to adjust the learning rate too.

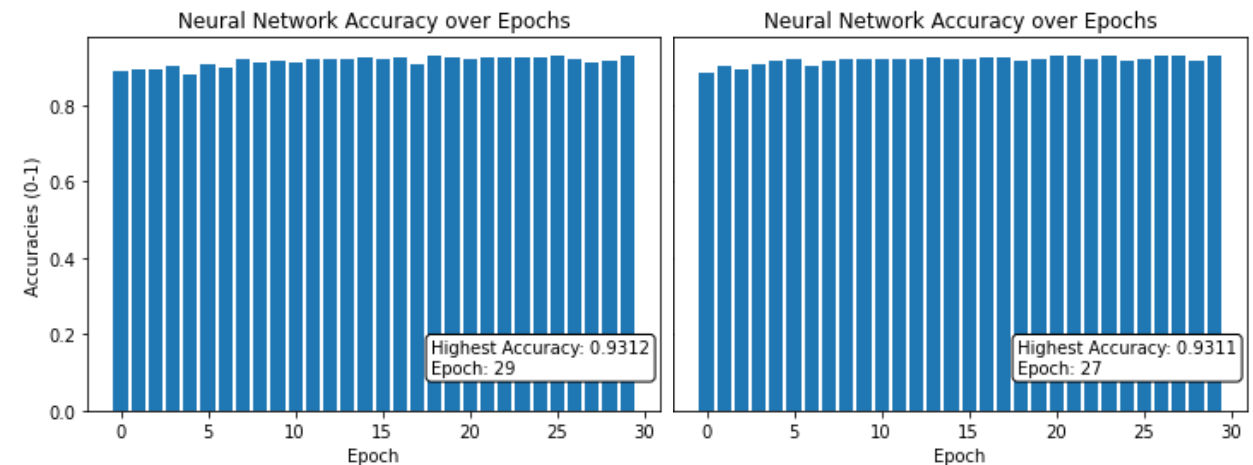


Fig 5. The neural networks in Fig 4., but with a learning rate of 6.0 instead of 3.0

The initial configuration of just three layers was still the most accurate (95.28%). For reference, the world record is 99.81%

Key Takeaways

- One can create a multi-level layer of perceptrons in order to simulate decision making, in the form of a neural network.
- The accuracy of a neural network depends heavily on the number of its layers, the density of its neurons, and parameters such as its learning rate and training batch size.

Reflection

This activity is a project I'll be proud to have done even long after I graduate. Even though the neural network I made was admittedly out of date, building it from scratch was not an easy task. Moreover, understanding multilayer perceptrons is a prerequisite to more sophisticated machine learning methods so this was a very insightful activity.

The module made by Ma'am Jing was very confusing for me so I didn't read it on my first pass on the topic. Instead, I watched reference [4] and then read the first two chapters of reference [3] in order to understand what was going on. These two made the index-chasing quite intuitive so I was then now able to read the module. Nevertheless, I still used the MNIST database as it is more standardized and the dataset is already prepared, why not use it?

I was able to make a neural network much more complex than what was required as I considered 784 inputs, and I was able to implement a customizable code. I was able to produce accurate results higher than 95% so I think I deserve a 100/100...

References

[1] Soriano, M. (2023). ML5 – Neural networks. App Physics 157. National Institute of Physics, UP- Diliman.

[2] Deng, L. (2012). The mnist database of handwritten digit images for machine learning research. IEEE Signal Processing Magazine, 29(6), 141–142.[3] McCulloch, Warren S., and Walter Pitts. "A logical calculus of the ideas immanent in nervous activity." The bulletin of mathematical biophysics 5, no. 4 (1943): 115-133

[3] Nielsen, M. (2015). Neural networks and deep learning. Determination Press.

[4] 3Blue1Brown. (2018). Neural networks – deep learning chapters 1 to 4. Uploaded in YouTube.com